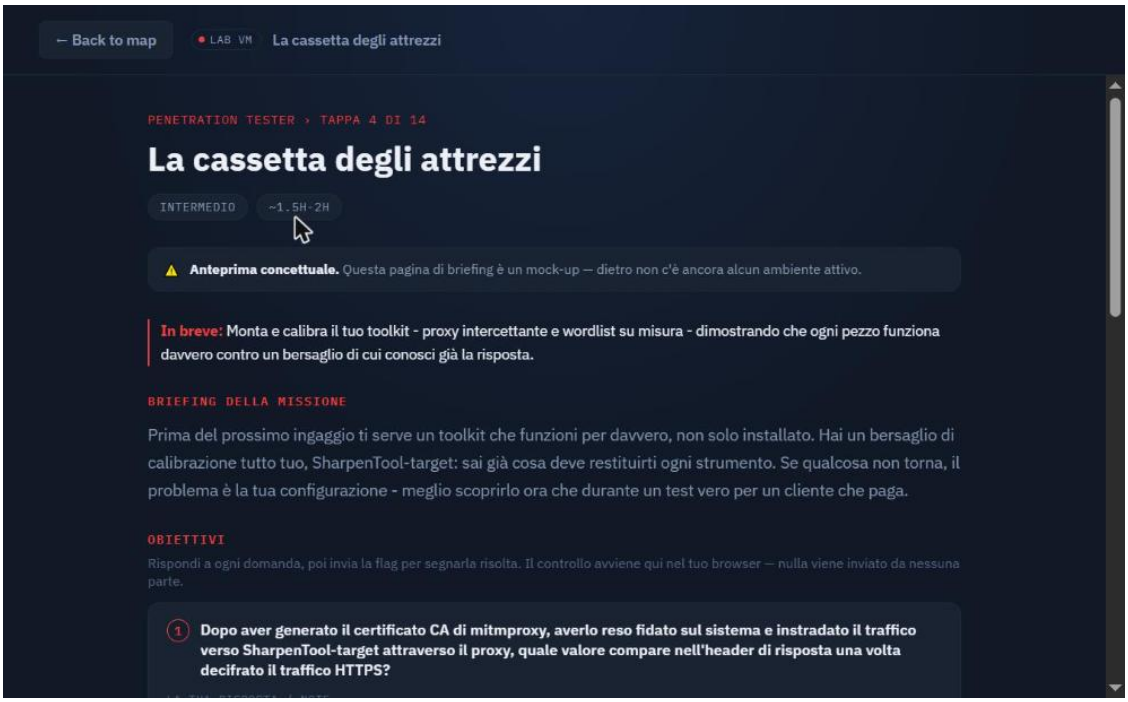
La cassetta degli attrezzi — Explanation

Ruolo: Penetration Tester — tier **va** (Vulnerability Analyst, CREST CPSA)
Lab id: the-toolbox · **Difficoltà**: Intermedio · **Durata stimata**: ~1.5h-2h
Lab component: lxr:net-perimeter (condiviso con "Primo contatto", "Come da manuale" e "Il linguaggio del rischio")

---

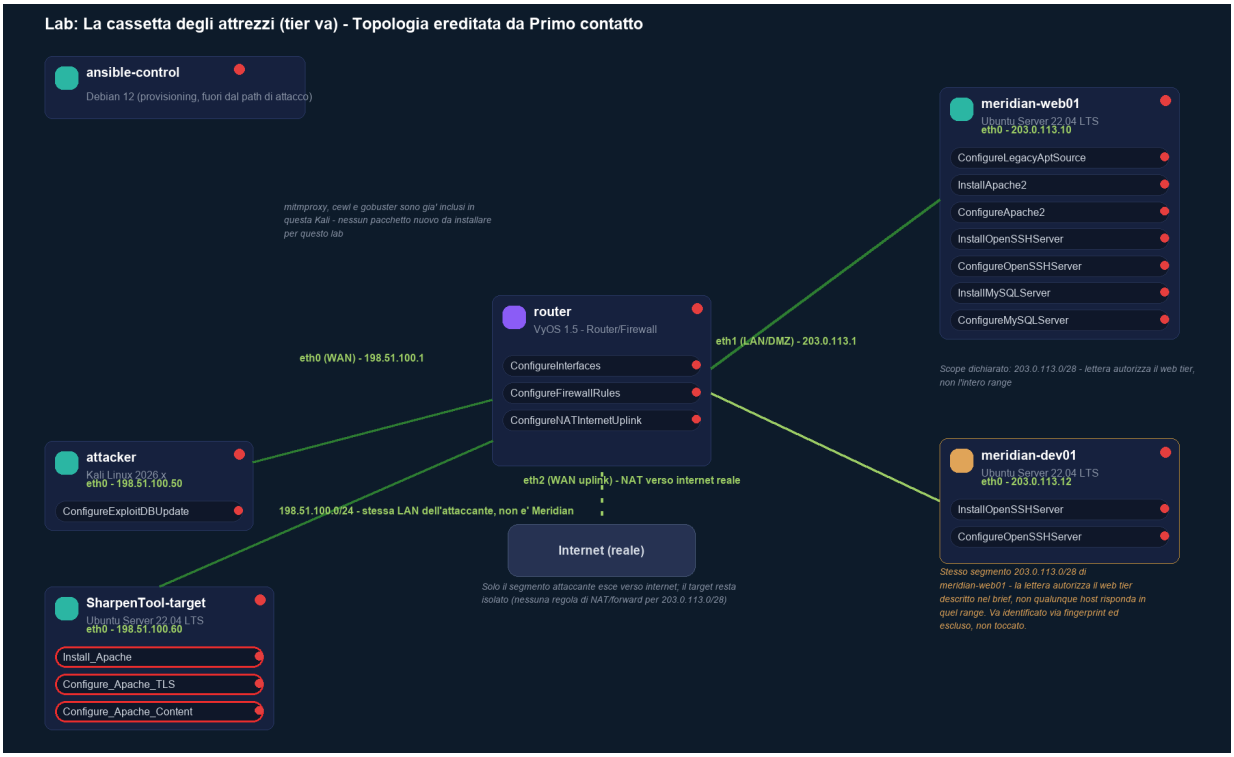
1. Schermata del lab (Avvia lab)



Screenshot reale della pagina di briefing così come appare oggi in piattaforma (versione HTB-restyled). La pagina è un mock-up concettuale: non c'è un ambiente live dietro. Come per i lab precedenti, nessun dato tecnico è già pronto da leggere nel testo: ogni finding deve emergere da un comando eseguito davvero (vedi sezione 7).

2. Infrastruttura

Questo lab riusa l'ambiente di "Primo contatto" (stesso `lxr:net-perimeter`): stessi host, stessa rete, stesso scope dichiarato verso Meridian (203.0.113.0/28). Nulla cambia su meridian-web01, meridian-dev01 o sul router — questo lab non tocca Meridian. L'unica aggiunta è una VM nuova sulla LAN dell'attaccante (198.51.100.0/24, la stessa dell'host `attacker`, non il segmento del cliente): `SharpenTool-target`, un bersaglio di calibrazione tutto tuo, con una risposta nota in anticipo, usato solo per verificare che il toolkit funzioni prima di puntarlo su un cliente vero. mitmproxy, cewl e gobuster sono già inclusi in questa Kali: nessun pacchetto viene installato su `attacker` per questo lab.



Stessa topologia di Primo contatto, con l'aggiunta di `SharpenTool-target` sulla LAN dell'attaccante — diagramma sorgente editabile in `the-toolbox-topology.drawio` (in questa stessa cartella): una copia di `docs/labs/risk-language/risk-language-topology.drawio` con la nuova card evidenziata in rosso (gli unici componenti nuovi per questo lab) e nessuna modifica al resto — meridian-web01 mantiene la configurazione già documentata nei lab precedenti.

3. Perché queste scelte

Storytelling

• **Episodio a sé**: la narrativa non fa alcun riferimento ai lab precedenti — nessun collegamento alla trama di Meridian. Solo l'infrastruttura di rete è condivisa (sezione 2); questo lab non tocca nemmeno Meridian, lavora esclusivamente sulla LAN dell'attaccante.

• **Calibrazione, non scoperta**: a differenza di tutti i lab precedenti, qui lo studente conosce già la risposta attesa di ogni strumento — l'obiettivo non è trovare qualcosa di nuovo, ma dimostrare che il proprio toolkit restituisce davvero quella risposta, prima di un ingaggio vero.

• **Nessun tool reinstallato inutilmente**: mitmproxy, cewl e gobuster sono già nella Kali di questo range (verificato su kali.org/tools/kali-meta/) — coerente con la stessa disciplina già applicata in Primo contatto e Come da manuale contro gli Install fittizi.

• **Nessuna exploitation**: generare un certificato, farlo fidare, instradare traffico, costruire una wordlist — tutte attività di setup/enumerazione, coerenti col tier va.

Infrastrutturali

• **Un'unica VM nuova, sulla LAN dell'attaccante**: non è un asset del cliente, non ha scope da verificare — deliberatamente separata da Meridian, per restare coerente con l'idea di "bersaglio di calibrazione proprio, non del cliente".

• **Risposte fissate in anticipo**: la versione di Apache, l'header di risposta e il path nascosto su `SharpenTool-target` sono valori noti e fissi — lo studente li confronta con l'output reale dei suoi strumenti, non li scopre da zero.

• **Nessuna modifica a Meridian**: meridian-web01, meridian-dev01 e il router restano esattamente come documentato nei lab precedenti.

4. Obiettivi

#	Domanda (flag)	Risposta
1	Dopo aver generato il certificato CA di mitmproxy, averlo reso fidato sul sistema e instradato il traffico verso SharpenTool-target attraverso il proxy, quale valore compare nell'header di risposta una volta decifrato il traffico HTTPS?	FLAG{TOOLKIT-VERIFIED}
2	Generando con cewl una wordlist su misura dal contenuto del sito di SharpenTool-target e usandola con gobuster per la content discovery, quale endpoint nascosto - mai linkato da nessuna pagina, introvabile con una wordlist generica - riesci a scoprire?	FLAG{ZEPHYRION-STATUS}

Mappatura ECSF: Task T8 "Deploy penetration testing tools and test programs" · Skill S8 "Use penetration testing tools effectively" · Knowledge K8 "Penetration testing tools".

Perché questi obiettivi

• **Flag 1 (mitmproxy)**: l'unico passaggio di setup davvero nuovo in tutto il percorso va — generare una CA, farla fidare dal sistema, instradarci il traffico — il punto dove la maggior parte dei principianti si blocca durante un ingaggio vero su un'applicazione web.

• **Flag 2 (cewl + gobuster)**: tecnica mai usata nei lab precedenti (wordlist costruita sul contenuto del target, non presa da una lista generica già pronta come in Come da manuale) — dimostra che una wordlist su misura trova cose che una generica non troverebbe mai.

• Nessuna flag richiede exploitation: sono entrambe verifiche di corretto funzionamento del toolkit, non ricerca di vulnerabilità.

• Difficoltà "Intermedio": ultimo lab del tier va prima del passaggio al tier pt, leggermente più impegnativo dei precedenti nonostante le sole due flag, per via dell'attrito reale di CA/trust e della generazione della wordlist.

5. Tool, OS e configurazioni

Componente	OS / Tecnologia	Install	Configure
SharpenTool-target (nuovo, calibrazione)	Ubuntu Server 22.04 LTS	Apache	Configure_Apache_TLS (HTTPS self-signed + header fisso), Configure_Apache_Content (sito statico)
attacker	Kali Linux 2026.x	- (mitmproxy, cewl, gobuster già in Kali)	-

Perché proprio questi tool/OS/configurazioni

• **mitmproxy**: tool reale e maturo (dal 2012, ancora attivamente mantenuto), già incluso in Kali — nessuna installazione, solo l'uso corretto: generazione della CA, trust, instradamento del traffico.

• **cewl**: già incluso in Kali, mai usato nei lab precedenti — genera una wordlist a partire dal contenuto reale di un sito, invece di affidarsi a una lista generica già pronta.

• **gobuster**: stesso tool già usato in Come da manuale, qui però con una wordlist autoprodotta invece di quella standard di dirb.

• **SharpenTool-target**: un host minimale ma riconoscibile come sito vero (home page + sezione About/Status), non una pagina di test vuota — con un header di risposta fisso e un path nascosto, entrambi valori noti in anticipo usati per verificare il toolkit.

6. Playbook section

Ruoli Ansible nuovi per questo lab, tutti su `SharpenTool-target` (VM nuova, nessun ruolo preesistente da modificare). Nessun ruolo nuovo su `attacker`: mitmproxy, cewl e gobuster restano esattamente come li installa la Kali di base, non c'è nulla da pre-configurare — generare la CA e usarla è compito dello studente durante il lab, non un'attività di provisioning. Percorso dei playbook effettivi: `catalogo/the-toolbox/`.

Componente	Ruolo	Tipo	Cosa fa / installa
SharpenTool-target	Install_Apache	Install	Installa Apache su una VM pulita, nessuna configurazione preesistente.
SharpenTool-target	Configure_Apache_TLS	Configure	Abilita HTTPS con un certificato self-signed e aggiunge l'header di risposta fisso X-Toolbox-Check, usato per verificare l'intercettazione di mitmproxy.
SharpenTool-target	Configure_Apache_Content	Configure	Pubblica il sito statico (home + sezione About/Status) e il path nascosto, mai linkato, usato per verificare la wordlist costruita con cewl.

7. Lab solution

1. Individuazione del bersaglio di calibrazione

SharpenTool-target non fa parte dello scope di Meridian: è sulla stessa LAN dell'attaccante (198.51.100.0/24), raggiungibile direttamente senza passare dal router.

```
nmap -sV 198.51.100.60
```

Risultato: 443/tcp aperta, Apache in HTTPS. Il bersaglio per tutto il lab è quindi `SharpenTool-target, 198.51.100.60`.

2. Flag 1 — setup del proxy intercettante (passo-passo)

2.1 Avviare mitmproxy per la prima volta

```
mitmproxy
```

Il comando apre un'interfaccia a schermo intero dentro il terminale (interfaccia testuale, si naviga con la tastiera) e, allo stesso tempo, avvia un server proxy in ascolto sulla porta 8080 di localhost. La prima volta che viene eseguito, mitmproxy genera da sé una propria CA (Certificate Authority) e salva i relativi file in `~/.mitmproxy/` — tra questi, `~/.mitmproxy/mitmproxy-ca-cert.pem` è il certificato pubblico della CA, quello da rendere fidato al passo successivo. mitmproxy resta aperto e in ascolto: va lasciato così e si lavora in un secondo terminale per i comandi seguenti.

2.2 Perché serve una CA e cosa succede senza

SharpenTool-target parla HTTPS: il traffico è cifrato end-to-end tra client e server, quindi un proxy normale vedrebbe solo byte cifrati, inutili. mitmproxy risolve il problema facendo da intermediario anche sulla cifratura: verso il client si presenta con un certificato che genera al volo per il sito richiesto, firmato dalla propria CA; verso il server reale apre una normale connessione TLS come farebbe qualunque client. Il traffico viene quindi decifrato due volte, una per lato, con mitmproxy che legge in chiaro nel mezzo. Perché questo funzioni senza errori, il client deve fidarsi della CA di mitmproxy esattamente come si fida di una CA pubblica riconosciuta: se si prova a instradare traffico nel proxy prima di questo passaggio, la richiesta fallisce con un errore di certificato non valido (self-signed / non riconosciuto), perché il client non ha alcun motivo di fidarsi di un certificato emesso da una CA che non conosce.

2.3 Rendere la CA fidata sul sistema (Kali/Debian)

```
sudo cp ~/.mitmproxy/mitmproxy-ca-cert.pem
/usr/local/share/ca-certificates/mitmproxy-ca-cert.crt
sudo update-ca-certificates
```

Il primo comando copia il certificato della CA nella cartella dove Debian/Kali tengono le CA aggiuntive (l'estensione va cambiata in `.crt` perché è quella che `update-ca-certificates` si aspetta di trovare lì). Il secondo comando ricostruisce il bundle di CA fidate di sistema includendo anche quella appena copiata: da questo momento ogni programma che si appoggia allo store di CA di sistema (incluso curl, se compilato con le librerie di sistema) considera valido qualunque certificato firmato da mitmproxy. In alternativa, senza toccare il sistema, si può dire al singolo comando curl di fidarsi solo di quel certificato con `--cacert ~/.mitmproxy/mitmproxy-ca-cert.pem` al posto dei due comandi sopra: più rapido per una verifica isolata, ma vale solo per quel comando, non per l'intero sistema.

2.4 Instradare il traffico attraverso il proxy

```
curl --proxy http://127.0.0.1:8080 https://198.51.100.60/
```

`--proxy http://127.0.0.1:8080` dice a curl di non contattare SharpenTool-target direttamente, ma di inviare la richiesta a mitmproxy, in ascolto in locale su quella porta (quella aperta al passo 2.1); è mitmproxy a occuparsi poi di contattare per davvero 198.51.100.60 e restituire la risposta a curl,

intercettando tutto nel mezzo. Con la CA fidata (passo 2.3) la richiesta va a buon fine e stampa l'HTML della pagina come se il proxy non ci fosse; senza, torna l'errore di certificato descritto al punto 2.2. Lo stesso principio vale per instradare un browser intero nel proxy, impostando lo stesso host/porta nelle sue impostazioni di rete, invece che per un singolo comando curl.

2.5 Leggere il flow intercettato nell'interfaccia di mitmproxy

Non appena la richiesta del passo 2.4 va a buon fine, nella finestra dove gira mitmproxy (passo 2.1) compare una nuova riga: un "flow", cioè una coppia richiesta/risposta intercettata, con metodo, URL e status code. Con le freccette si seleziona quella riga e si preme Invio per aprirne il dettaglio; dentro il dettaglio si passa tra le sezioni Request e Response con Tab, e dentro la sezione Response sono elencati tutti gli header della risposta decifrata — compreso `X-Toolbox-Check: TOOLKIT-VERIFIED`, un header normalmente invisibile perché viaggia cifrato dentro l'HTTPS, leggibile qui solo perché mitmproxy l'ha decifrato per noi. Con q si torna alla lista dei flow, con q di nuovo (o Ctrl+C) si esce dal programma quando il lab è completato → **FLAG{TOOLKIT-VERIFIED}**.

3. Flag 2 — wordlist su misura con cewl e gobuster (passo-passo)

3.1 Generare la wordlist con cewl

```
cewl https://198.51.100.60/ -w toolbox-wordlist.txt
```

cewl (Custom Word List generator) visita l'URL indicato, ne legge il testo della pagina (titoli, paragrafi, parole usate nel contenuto) e con `-w toolbox-wordlist.txt` salva ogni parola unica trovata in quel file, una per riga: il risultato è una wordlist pensata apposta per questo sito, non una lista generica. Sulla pagina About/Status di SharpenTool-target compare il nome di un modulo interno, "Zephyrion": cewl lo raccoglie come una qualunque altra parola del testo e lo inserisce nel file — una wordlist generica come quella di dirb usata in "Come da manuale" non conterrebbe mai questa parola, perché non è un termine comune, è specifico di questo sito.

3.2 Content discovery con gobuster usando la wordlist appena creata

```
gobuster dir -u https://198.51.100.60/ -w toolbox-wordlist.txt -k
```

`dir` seleziona la modalità di gobuster che cerca percorsi/directory su un sito web (bruteforce di URL). `-u` indica il bersaglio, `-w` la wordlist da provare — qui quella appena generata con cewl, non una wordlist standard. `-k` dice a gobuster di ignorare l'errore di certificato self-signed di SharpenTool-target (lo stesso tipo di certificato non riconosciuto visto al punto 2.2, ma qui non interessa intercettare il traffico, solo raggiungere il sito, quindi si può semplicemente ignorare l'avviso invece di passare da mitmproxy). gobuster prova ogni parola della wordlist come se fosse un percorso (es. `/zephyrion/`, `/status/`...) e segnala quelli che il server riconosce.

3.3 Leggere l'output

gobuster stampa una riga per ogni percorso trovato, con il relativo status code (es. `Status: 200`). Tra i risultati compare `/zephyrion-status/`, con status 200: un endpoint mai linkato da nessuna pagina visibile del sito, irraggiungibile con una wordlist generica, scoperto solo perché la wordlist costruita al punto 3.1 conteneva proprio il termine "Zephyrion" letto dal contenuto reale del sito → **FLAG{ZEPHYRION-STATUS}**.

4. Toolkit verificato

Entrambe le flag confermano che scanner, proxy e wordlist restituiscono esattamente quanto atteso contro un bersaglio di cui già conoscevi la risposta: il toolkit è pronto per essere usato su un target vero, non solo installato.